

Link to our GitHub repository:

https://github.com/juliaaalopez/irwa_search_engine_G_011.git

Group number: 11

Group members: Júlia López, Isabel Expósito, Margarita Crespo, Dídac Hierro

Final Project Part 2: Indexing and Evaluation

This second part of the project focuses on building the core components of our retrieval system. The goal is to allow efficient and accurate retrieval of relevant products based on textual queries. In order to do this, we first construct an inverted index, the fundamental data structure that supports fast search operations. Later, we will extend this part to include ranking algorithms (e.g., TF-IDF) and evaluation metrics to assess retrieval performance.

The starting point for this part was the cleaned and preprocessed dataset generated in Part 1. It should be noted that while doing this part of the project, we encountered some minor issues that required modifying parts of the previous code to ensure compatibility and completeness.

First, in order to work with the cleaned dataset, we added a line of code to export it as a .csv file, making it easier to reuse the data across different parts of the project. Second, during this stage, we realized that, in order to propose the text queries, we need to take into account the most frequent terms of both the *processed_text* (*title* and *description*) and the *attributes*, but in our analysis we only considered the former, hence we added the latter.

Part 1: Indexing

1.1 Building the inverted index

Before building the index, we first evaluated the proportion of missing values in the dataset and decided how to handle them.

Data columns (total 14 columns):				
#	Column	Non-Null Count		Dtype
---	-----	-----	-----	-----
0	pid	28080	non-null	object
1	url	28080	non-null	object
2	processed_text	28080	non-null	object
3	title	28080	non-null	object
4	description	16927	non-null	object
5	brand_facet	26071	non-null	object
6	category_facet	28080	non-null	object
7	subcategory_facet	28080	non-null	object
8	seller_facet	26437	non-null	object
9	discount	27225	non-null	float64
10	selling_price	28078	non-null	float64
11	actual_price	27303	non-null	float64
12	average_rating	25819	non-null	float64
13	attributes	28080	non-null	object

Fig 1. Output of `df.info()` of our dataset

Given the detected missing values, we treated them differently according to the type of data.

- For *description*, *brand_facet*, and *seller_facet*, we replaced the null entries with the string *unknown* since it keeps the variable searchable and consistent.
- For the numerical missing values of *discount*, *selling_price*, *actual_price*, and *average_rating*, we input a 0 value, allowing these fields to remain usable in quantitative analyses and preventing computational errors.

We also realized that since *processed_text* is the combination of *title* and *description*, and *description* contains null entries, then *processed_text* may contain some unwanted tokens, such as *none* or *nan*. In order to keep the variable accurate, we safely filter out these unwanted tokens.

Once we ensured that all the dataset was complete, we started the implementation of the inverted index.

To build this structure, we first carried out a token combination of both variables that contained product information: *processed_text* and *attributes*. This will ensure that all descriptive information is represented in the same place and becomes searchable.

We then created a function **build_inverted_index(df)**, which initializes a dictionary and iterates over all products in our dataset. For each token, the function adds the product's *pid* to the set of documents associated with that token. This results in a structure where each term maps to the list of product IDs that contain it.

```
solid: ['4hWtdShnw%3D%3D', 'BDAFYGRSSHGHBBYE', 'BRFEWB5RPBDW9DKP', 'BRFEX35MBHYTBYGC', 'BRFEZEF49JAWFCMD']
women: ['ARME8P6GZGFHAUPX', 'BDAFSFMYKJDGSXUG', 'BDAFT7NGPTHYGVJR', 'BDAFT8JHVBQ6ZT3F', 'BDAFT8JQFNVHGBVH']
multicolor: ['ARME8P6GFSXGE9AW', 'ARME8P6GJQ8PZBE6', 'ARME8P6GXFED7RJZ', 'ARME8P6GZGFHAUPX', 'BDAFSFMYKJDGSXUG']
track: ['BXRFGGX685HKER8Y', 'BXRFGYQTPZNHEWFD', 'BXRFGYQUTHXJKHHP', 'BXRFGYQUZH5DCABH', 'BXRFGYRYGUURZGDG']
pant: ['BXRFGGX685HKER8Y', 'BXRFGYQTPZNHEWFD', 'BXRFGYQUTHXJKHHP', 'BXRFGYQUZH5DCABH', 'BXRFGYRYGUURZGDG']
yorker: ['TKPFCZ9DYU33FFXS', 'TKPFCZ9E2UC3DR3F', 'TKPFCZ9EA7H5FYZH', 'TKPFCZ9ECDYYDNKA', 'TKPFCZ9EFK9DNWDA']
trackpant: ['CRGFGQRNCB7QVEME', 'CRGFGQS63JKJZM34', 'TKPEFVQAPRHKVJHJ', 'TKPEFVQB8Z65YCYQ', 'TKPEFVQBA9FJA8WX']
rich: ['CRGFB6UNUPXTTY7F', 'CRGFDWCNXYTB8SAM', 'CTPFVZD8CNSZ3AMR', 'CTPFVZEYHCRQ27Y2', 'CTPFVZYFR8KGYBJ']
comb: ['BRFEWB5RPBDW9DKP', 'BRFEX35MBHYTBYGC', 'BRFEZEF49JAWFCMD', 'BRFEZWU9FCHYPJDU', 'BRFFYNFYDDNAUDZZ']
cotton: ['4hWtdShnw%3D%3D', 'BDAFSFMYKJDGSXUG', 'BDAFUBD2EJHFCRNC', 'BRFEGM47AKYADGCU', 'BRFEP9YFZF3ZEBSY']
```

Fig 2. Output of the inverted index's format.

Finally, to avoid rebuilding the index each time, we serialized it into a JSON file saved in the data directory of our project repository.

1.2 Test Queries

The next step was to create a set of 5 queries we would later use in order to test our search engine to see whether the index we previously created retrieved the right products and how relevant they were.

To choose the queries, we took a look at the exploration data analysis we did in part 1 of the project, and by looking at the word cloud of the most frequent terms of the dataset, we came up with 5 different queries. We tried to include terms from the descriptions, titles, brands, categories, etc (not all from the same fields).

Since we defined our dataset as preprocessed tokens, we must apply the same text preprocessing steps to our queries, since we need to ensure that query words and indexed

words are represented in the same form. Hence, we applied both **setup_processing_tools()** and **preprocess_query()**, which is the same as **preprocess_text()** from part 1.

Hence, at this point, we had 5 test queries aligned with the just-built inverted index.

1.3 Result Ranking

Now, the only step left to have a working search engine was to implement a ranking function to order the results by relevance, given an input query.

For that task, we implemented the TF-IDF algorithm:

First, we computed the DF scores and IDF scores for the terms in the inverted index. Then, we computed the joint TF-IDF score using the dot product of the log-transformed term frequency and the logarithm of N (number of documents in the dataset) over the document frequency.

$$w_{i,j} = \begin{cases} (1 + \log f_{i,j}) \times \log \frac{N}{df_i} & \text{if } f_{i,j} > 0 \\ 0 & \text{otherwise} \end{cases}$$

Fig 3. TF-IDF score formula

For this, we defined **calculate_log_tf()** and **calculate_tfidf_L2_norm()**. The latter function, additionally, computes the L2-norm (length) of the computed TF-IDF vector, which is later used to compute cosine similarity.

The last step was to use these computed scores to rank the results in decreasing order of relevance (higher score equals higher relevance). The function **search_tfidf()** computes and returns the ordered ranking of the k most relevant products given an input query. To do it, it first identifies documents that match the query by intersecting the posting lists of the query terms, meaning that a document must contain all query terms to be considered.

This step significantly reduces the number of documents that need to be scored, improving efficiency by only considering products that are guaranteed to contain every term in the search query. If no product fulfills this condition, we just say that no document contains all query terms and don't return any result.

Once we are left with only the documents that contain all the query terms, the function computes the cosine similarity of each of them to the input query by using the previously defined scores. Finally, it ranks the results by decreasing similarity and returns the top k (we use k = 10).

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

Fig 4. Cosine similarity formula

Part 2: Evaluation

2.1 Evaluation start, metric functions

Now that we have our ranking function based on TF-IDF, we want to know how good the results returned by our search engine are. That is, we want to compute metrics that measure the quality of our retrieval results.

We rely on two key elements for each query:

- the ground truth (y_true), an array of ground-truth relevance labels (1=relevant, 0=not relevant)
- the predicted scores (y_score), which are the similarity scores obtained from our ranking model

We have implemented seven evaluation metrics:

- Precision@K: We measure the fraction of relevant items among the top-K retrieved results. High precision indicates that there are fewer irrelevant items in the first K results.

$$P@K = \frac{1}{K} \sum_{i=1}^K rel(i)$$

- Recall@K: of all the relevant documents in the dataset, recall measures the proportion of relevant documents retrieved within the top K. High recall means that our system is able to find most of the relevant documents available.

$$R@K = \frac{\sum_{i=1}^K rel(i)}{R}$$

- Average Precision@K: computes the average of the precision values at each rank position where a relevant document appears. It rewards systems that rank relevant documents higher in the results list.

$$AP@K = \frac{1}{R} \sum_{i=1}^K P(i) \cdot rel(i)$$

- F1-Score@K: is the harmonic mean of precision and recall at K. This balances both metrics in a single score. A high F1 means that the system retrieves many relevant documents while maintaining them high in the ranking.

$$F1@K = \frac{2 \cdot P@K \cdot R@K}{P@K + R@K}$$

- Mean Average Precision: represents the mean of the Average Precision@K across all queries. This provides a global performance score over all queries. This metric is really important since it balances precision, recall, and ranking order.

$$MAP = \frac{1}{Q} \sum_{q=1}^Q AP_q$$

- Mean Reciprocal Rank: computes how soon the first relevant document appears in each query's ranked list. A high value of this metric indicates that relevant results appear very early in the ranking, which improves user satisfaction.

$$MRR = \frac{1}{Q} \sum_{q=1}^Q \frac{1}{\text{rank of first relevant doc}}$$

- Normalized Discounted Cumulative Gain: measures ranking quality while discounting lower ranks; that is, the relevant results placed higher contribute more to the score. The ideal DCG (IDCG@K) corresponds to the best possible ranking within the top K.

$$DCG@K = \sum_{i=1}^K \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

- Precision and Recall Curve: a graphical representation of the precision and recall that allows us to visualize the trade-off between both metrics.

2.2 Metrics

For this part, we applied all the evaluation metrics implemented in section 2.1 to the two predefined queries, using the relevance judgments provided in *validation_labels.csv*.

The queries defined for this evaluation were:

- Query 1: "women full sleeve sweatshirt cotton"
- Query 2: "men slim jeans blue"

We started by loading the *validation_labels.csv* file, which contains the manually assigned relevance labels for each product and query. Each label takes a binary value: 1 if the product is relevant to the query, and 0 if it is not. In order to correctly define the functions above, we took this into consideration.

Then, we generated a new dataframe (*sys_res*) containing the results of our search system. For each query, we executed the function **search_tfidf()**, which retrieves the top-10 products along with their predicted relevance scores (*y_score*). Each row in the dataframe represents a tuple (*query_id*, *pid*, *y_score*), identifying the query, product, and its TF-IDF similarity score.

Next, we merged the system results dataframe with the relevance labels dataframe using the fields *query_id* and *pid*. This merge produced a unified evaluation dataframe (*compare*) that aligns our system's predicted scores with the ground truth labels for each query.

We also used the function **check_missing_scores()** to confirm that there were no missing or undefined values in the merged dataframe.

Finally, we computed the metrics for each of the queries using the ground truth and the predicted relevance arrays, and we rounded all results to 3 decimals.

Query	P@10	R@10	AP@10	F1@10	MRR@10	NDCG@10
0	1	1.0	0.769	1.000	0.87	1.000
1	2	0.3	0.300	0.317	0.30	0.244

Fig 5. Output of metrics applied in example queries

For Query 1 (indexed as query 0 in our image), all metrics are very high. Both P@10, AP@10, and NDCG@10 reached a perfect score of 1.000, and Recall@10 is also strong with a value of 0.769. This means that almost all retrieved products are relevant and ranked correctly at the top of the list. We can also see how the F1-Score and MRR values confirm this ‘trend’, showing that the system identifies the relevant items early in the ranked list.

In contrast, Query 2 shows noticeably lower values.

P@10, Recall@10, and F1@10 are around 0.3, while AP@10 and NDCG@10 fall below 0.4. These results suggest that the system retrieved fewer relevant jeans among the top-ranked results, and the ranking order was less accurate.

This difference may be due to less overlap between the query terms and the product metadata. For example, some jeans might use alternative color terms which we did not think of when defining our color map in part 1, such as ‘denim’, or omit the adjective ‘slim’.

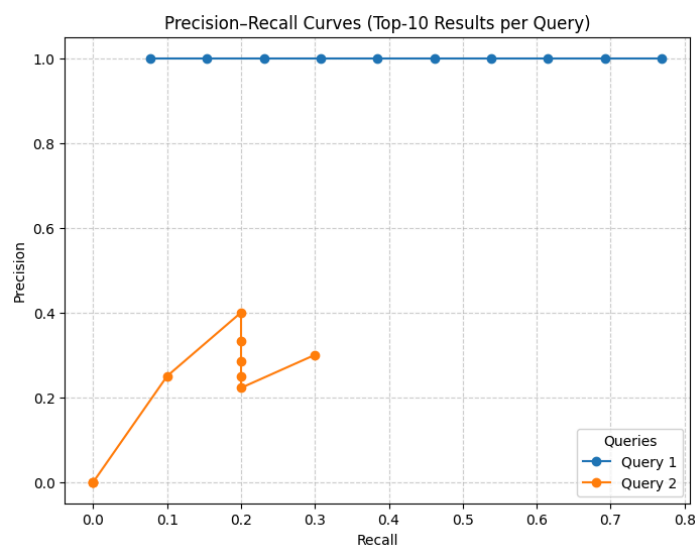


Fig 6. Precision and recall curves for queries 1 and 2

All this reasoning is backed up by the precision and recall curve for both Query 1 and 2. We can confirm that query 1 achieves consistently high retrieval quality while Query 2’s performance drops, likely due to weaker lexical overlap between the query and the document term.

These results are consistent with what we expected from our indexing and preprocessing steps in Part 1. Since TF-IDF does not include semantic understanding or synonym expansion (even though we included some for the color map), some terms that are conceptually related (‘jeans’ vs. ‘denim’) are not matched, reducing effectiveness for certain queries like the second one.

2.3. Analysis of the current system, manual research

QUERY 1: 'blue shirt round neck machine wash'

The user is asking for a

- shirt
- blue colored
- round neck
- that we can wash in the washing machine

→ The user didn't specify "solid" or "plain." Printed blue shirts still fulfill the core need.

Rank	Title	Brand	Reasoning	Label
1	Printed Men Round Neck Blue T-Shirt	rose_we	We put it as 1 since 3/4 characteristics are in the title	1
2	Printed Men Round Neck Blue T-Shirt	xi	We put it as 1 since 3/4 characteristics are in the title	1
3	Solid Men Round Neck Blue T-Shirt	axma	We put it as 1 since 3/4 characteristics are in the title	1
4	Solid Men Round Neck Blue T-Shirt	axma	We put it as 1 since 3/4 characteristics are in the title	1
5	Printed Men Round Neck Blue T-Shirt	yellowvib	We put it as 1 since 3/4 characteristics are in the title	1
6	Printed Women Round Neck Blue T-Shirt	xi	We put it as 1 since 3/4 characteristics are in the title	1
7	Printed Men Round Neck Blue T-Shirt	xi	We put it as 1 since 3/4 characteristics are in the title	1
8	Printed Men Round Neck Blue T-Shirt	xi	We put it as 1 since 3/4 characteristics are in the title	1
9	Printed Women Round Neck Blue T-Shirt	steenb	We put it as 1 since 3/4 characteristics are in the title	1
10	Printed Men Round Neck Blue T-Shirt	steenb	We put it as 1 since 3/4 characteristics are in the title	1

y_true = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

QUERY 2: 'solid men blue trackpants'

The user is asking for:

- trackpants
- solid (no prints)
- men
- blue

Rank	Title	Brand	Reasoning	Label
1	Solid Men Blue Track Pants	mountain_colou	We put it as 1 since 4/4 characteristics are in the title	1
2-5	Solid Men Blue Track Pants	pu	We put it as 1 since 4/4 characteristics are in the title	1
6	Solid Men Blue Track Pants	jack_and_har	We put it as 1 since 4/4 characteristics are in the title	1
7-8	Solid Men Dark Blue Track Pants	onei	We put it as 1 since 4/4 characteristics are in the title	1
9	Solid Men Dark Blue Track Pants	jack_and_har	We put it as 1 since 4/4 characteristics are in the title	1
10	Solid Men Black, Dark Blue Track Pants	onei	We put it as 1 since 4/4 characteristics are in the title (penalize if we assume that it containing black as well makes it non relevant)	0

y_true = [1, 1, 1, 1, 1, 1, 1, 1, 1, 0]

QUERY 3: 'full sleeve casual shirt cotton'

The user is asking for:

- full sleeve
- shirt
- casual
- cotton

→ No print specified, so let's penalize if printed or similar appears, since the user is probably looking for a basic piece

Rank	Title	Brand	Reasoning	Label
1-3	Women Regular Fit Solid Casual Shirt	sora	We put it as 1 since 2/4 characteristics are in the title	1
4	Men Regular Fit Printed Casual Shirt	sora	We put it as 0 since 2/4 characteristics are in the title, but printed	0
5	Women Regular Fit Solid Casual Shirt	sora	We put it as 1 since 2/4 characteristics are in the title	1
6	Men Regular Fit Solid Casual Shirt	sora	We put it as 1 since 2/4 characteristics are in the title	1
7-8	Women Regular Fit Printed Casual Shirt	sora	We put it as 0 since 2/4 characteristics are in the title, but printed	0

9-10	Men Regular Fit Printed Casual Shirt	sora	We put it as 0 since 2/4 characteristics are in the title, but printed	0
------	--------------------------------------	------	--	---

y_true = [1, 1, 1, 0, 1, 1, 0, 0, 0, 0]

QUERY 4: 'women polo t-shirt'

The user is asking for:

- women
- polo t-shirt

→ no print specified and the fact that it is a polo reduces the options enough (there are not many polo types), so let's not penalize if printed or similar appears

Rank	Title	Brand	Reasoning	Label
1	Solid Women Polo Neck Blue T-Shirt	scott_internation	We put it as 1 since 2/2 characteristics are in the title	1
2	Solid Women Polo Neck Black T-Shirt	scott_internation	We put it as 1 since 2/2 characteristics are in the title	1
3	Solid Women Polo Neck Dark Blue T-Shirt	got	We put it as 1 since 2/2 characteristics are in the title	1
4-5	Printed Women Polo Neck Blue T-Shirt	yellowvib	We put it as 1 since 2/2 characteristics are in the title but printed	1
6	Solid Women Polo Neck Black T-Shirt	scott_internation	We put it as 1 since 2/2 characteristics are in the title	1
7	Solid Women Polo Neck Blue T-Shirt (Pack of 3)	pol	We put it as 1 since 2/2 characteristics are in the title	1
8	Printed Women Polo Neck Blue T-Shirt	true_bl	We put it as 1 since 2/2 characteristics are in the title	1
9	Printed Women Polo Neck Blue T-Shirt (Pack of 3)	yellowvib	We put it as 1 since 2/2 characteristics are in the title	1
10	Solid Women Polo Neck White T-Shirt	u.s._polo_ass	We put it as 1 since 2/2 characteristics are in the title	1

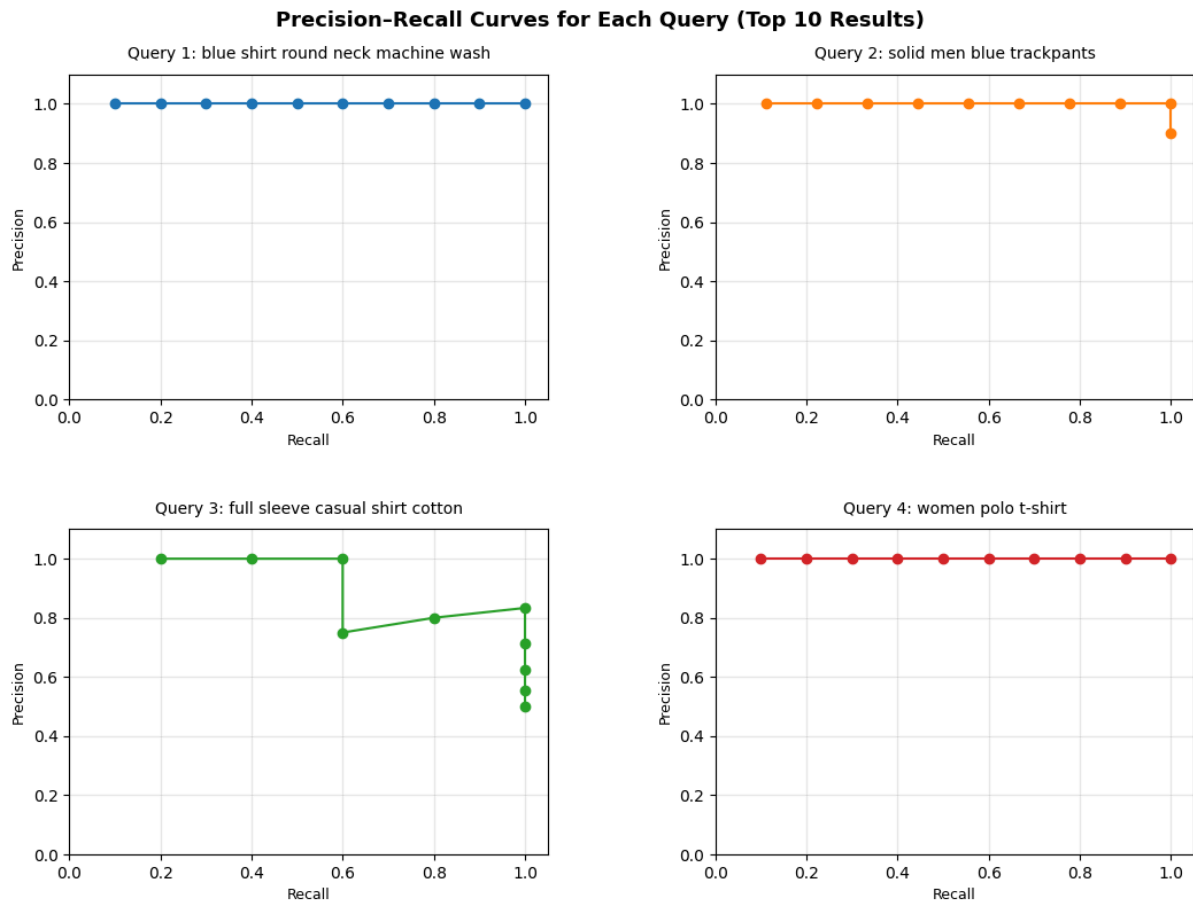
y_true = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

QUERY 5: 'white shirt cotton'

The user is asking for:

- white
- shirt
- cotton

No print specified, although it is not needed to apply print since all results are t-shirts when shirts are different, so $y_true = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$.



The figure above shows the Precision-Recall (PR) curves for the five custom queries defined in this part. Each plot represents how precision changes as recall increases for the top 10 retrieved results. We can observe that both Query 1 and Query 4 achieve perfect precision across all recall levels, which confirms that all top results are highly relevant. For Query 2 is practically the same; it maintains near-perfect precision. On the other hand, Query 3 shows a slight drop in precision as recall increases, indicating some non-relevant results appear in lower ranks. Finally, for Query 5, since it has no relevant result, the curve would be flat near the bottom, which we decided not to include.

Some limitations and possible improvements

After analyzing all queries, we observed that our TF-IDF system performs well when the query terms directly match the product titles, but it also shows some expected limitations.

Some of them might be:

- **Exact Word Matching:** TF-IDF only matches identical terms, so it fails to capture semantic relations (jeans vs denim), so we could use some synonym expansion.
- **Equal field weight:** We treat all text fields equally, even though the title is more informative than the brand, for example, so we could apply some field-specific weights in the future.

- Ranking the bias: Because some documents with repeated queries may appear at the top even if they are not fully relevant, so in the future we should think about a better normalization.

References

To develop this part of the project, we used Gemini AI as a code assistant to correct the code syntax.

We also used ChatGPT to review the report and Readme.md and brainstorm approaches.